

self-attention

Что такое Self-Attention?

Self-Attention - механизм, который позволяет каждому токenu определить, какие другие токены наиболее важны для понимания его собственного смысла.

Например:

```
Мальчик бросил мяч собаке
```

Если модель обрабатывает слово **"бросил"**, то ей полезно посмотреть на:

- мальчик - кто бросил
- мяч - что бросил
- собаке - кому бросил

Главная идея Self-Attention:

Каждый токен может посмотреть на все остальные токены и решить, насколько они ему полезны.

Эмбединги

После токенизации каждое слово представляется вектором.

Например:

```
X1 = embedding("Мальчик")  
X2 = embedding("бросил")  
X3 = embedding("мяч")  
X4 = embedding("собаке")
```

Если размерность эмбединга равна 4096, то

```
Xi ∈ ℝ4096
```

Каждый эмбединг содержит некоторое числовое представление смысла слова.

Однако использовать эмбединг напрямую неудобно, поэтому Transformer строит три новых представления.

Query, Key и Value

Из каждого эмбединга вычисляются три новых вектора.

```
Q = XWQ  
K = XWK  
V = XWV
```

где

X - эмбединг токена

W_Q - обучаемая матрица весов Query

W_K - обучаемая матрица весов Key

W_V - обучаемая матрица весов Value

Важно понимать разницу.

Матрицы весов:

W_Q

W_K

W_V

одни и те же для всех токенов слоя.

А вот результаты

Q_1

Q_2

Q_3

...

K_1

K_2

...

V_1

V_2

...

разные, потому что эмбединги разные.

Интуитивный смысл

Можно представить роли так.

Query

| Что я сейчас ищу?

Key

| Насколько я подхожу под этот запрос?

Value

| Если меня выбрали, какую информацию я передам?

Именно Value хранит информацию, которая попадет в итоговый результат.

Размерности

Пусть

n - количество токенов
 d_k - размерность Key
 d_v - размерность Value

Тогда

$Q \in \mathbb{R}^{n \times d_k}$
 $K \in \mathbb{R}^{n \times d_k}$
 $V \in \mathbb{R}^{n \times d_v}$

Например

$n = 4$
 $d_k = 128$
 $d_v = 128$

Получаем

$Q = (4 \times 128)$
 $K = (4 \times 128)$
 $V = (4 \times 128)$

Что такое d_k ?

d_k - это размерность каждого вектора Key.

Если

$K_i = [\dots, 128 \text{ элементов } \dots]$

то

$d_k = 128$

В простой модели часто

$d_{\text{model}} = d_k = d_v$

Но вообще они не обязаны совпадать.

Размерность Q , K и V задается матрицами

W_Q
 W_K

$W \cdot V$

Вычисление Attention Score

Каждый Query сравнивается со всеми Key.

Например

$$Q_2 \cdot K_1$$

$$Q_2 \cdot K_2$$

$$Q_2 \cdot K_3$$

$$Q_2 \cdot K_4$$

То есть токен "бросил" сравнивается со всеми токенами предложения.

Важно.

Сравниваются не отдельные элементы вектора.

Сравниваются целые векторы.

Если предложение содержит n токенов, то вычисляется

$$n^2$$

скалярных произведений.

Именно поэтому сложность Self-Attention равна

$$O(n^2)$$

Почему именно Q и K?

Query можно понимать как запрос.

Key - как описание того, насколько токен подходит под этот запрос.

Поэтому сравниваются именно

$$Q \cdot K$$

а не

$$Q \cdot Q$$

или

$$K \cdot K$$

Матричная запись

Все вычисления сразу записываются как

$$QK^T$$

Размерности

$$Q = (n \times d_k)$$

$$K^T = (d_k \times n)$$

Следовательно

$$(n \times d_k)$$

×

$$(d_k \times n)$$

=

$$(n \times n)$$

Получается матрица оценок.

Например

	K_1	K_2	K_3	K_4
Q_1	1.2	0.8	2.1	0.4
Q_2	0.3	1.5	0.6	0.7
Q_3	...			
Q_4	...			

Элемент

$$(i, j)$$

равен

$$Q_i \cdot K_j$$

Масштабирование

После вычисления оценок выполняется

$$\frac{QK^T}{\sqrt{d_k}}$$

Зачем?

Если размерность вектора большая, то скалярное произведение становится значительно больше.

Большие значения приводят к тому, что Softmax становится слишком "резким".

Например

```
[310, 305, 298]
```

после Softmax почти превращаются в

```
[1, 0, 0]
```

Из-за этого ухудшается обучение.

Поэтому оценки масштабируются.

Softmax

После масштабирования применяется

```
Softmax
```

Размерность не меняется.

Было

```
(n × n)
```

Стало

```
(n × n)
```

Но теперь каждая строка представляет распределение вероятностей.

Например

До Softmax

```
[2.1, 0.8, 1.4, -0.2]
```

После

```
[0.54, 0.15, 0.26, 0.05]
```

Сумма каждой строки равна

```
1
```

Каждая строка отвечает на вопрос

На какие токены смотрит текущий токен?

Зачем нужен Value?

После Softmax мы знаем только

Кому доверять.

Но мы еще не получили никакой информации.

Например

Мальчик - 0.2

Бросил - 0.1

Мяч - 0.6

Собаке - 0.1

Эти числа сами по себе ничего не содержат.

Они лишь говорят

Кого следует слушать.

Информация хранится в Value.

Поэтому вычисляется

$\text{Softmax}(QK^T / \sqrt{d_k})V$

Размерности

$(n \times n)$
 $\times$
 $(n \times d_v)$
 $=$
 $(n \times d_v)$

Если

$n = 4$
 $d_v = 128$

то

(4×4)
 $\times$
 (4×128)

=
(4 × 128)

Получается новый вектор для каждого токена.

Для второго токена это выглядит как

$0.2V_1$
+
 $0.1V_2$
+
 $0.6V_3$
+
 $0.1V_4$

То есть модель берет информацию от каждого токена, но в разной степени.

Полная формула Self-Attention

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Что важно запомнить

- Каждый токен имеет собственные Q, K и V.
- W_Q , W_K и W_V - это обучаемые матрицы весов.
- Каждый Query сравнивается со всеми Key.
- Ни одна пара токенов не пропускается.
- После QK^T получается матрица размера $n \times n$.
- Softmax превращает оценки в распределение внимания.
- Value хранит информацию, которую токен может передать другим.
- Итоговый вектор является взвешенной суммой всех Value.
- После умножения на V размерность снова становится $n \times d_v$.

Собственные выводы

1. Размерность после умножения на V возвращается к размерности представлений токенов, поэтому результат можно передавать в следующий слой Transformer.
2. Каждая строка матрицы Attention отвечает за один токен и показывает, насколько он "смотрит" на остальные токены.
3. В Self-Attention сначала определяется **кому доверять** (Q и K), и только потом извлекается **какую информацию получить** (V).

4. Если предложение содержит n токенов, то каждый Query сравнивается с каждым Key. Поэтому вычисляются все возможные пары токенов, без исключений.
5. Value не участвует в вычислении внимания. Он используется только после того, как модель определила, какие токены наиболее важны.
6. Query, Key и Value являются разными представлениями одного и того же эмбединга, каждое из которых отвечает за свою роль в механизме Attention.